ARELORIA / WASD ENTWICKLER-MANUSKRIPT-MANIFEST PHASE 2

Roadmap, Onboarding, Implementierungsstand und
Leitfaden für spätere Entwicklerinnen, Entwickler und Agenten

Quellbasis	ZIP-Snapshot: Wasd-main (2).zip
Repo-Ziel	thosu87-svg/Wasd (Live-URL genannt, Webabruf jedoch nicht verlässlich reproduzierbar)
Snapshot-Datum	17.03.2026 laut ZIP-Inhalt / Dateiständen
Dokumentzweck	Phase-2-Handbuch für Devs, Leads, Agenten und spätere Handoffs
Dokumentstatus	Architektur- und Arbeitsmanuskript, kein öffentliches Release-Zertifikat

Methodischer Hinweis: Dieses Phase-2-Manuskript basiert auf der gelieferten ZIP-Momentaufnahme und deren Dateiinhalten. Aussagen über den Umsetzungsstand wurden nur dann als „stark“ oder „brauchbar“ bewertet, wenn im Snapshot sowohl Struktur als auch konkrete Laufzeit- oder Datenpfade sichtbar waren.

1. Executive Summary

Der Snapshot zeigt ein ungewöhnlich breites Hybrid-Repository: Es enthält nicht nur Konzept- und Handoff-Dokumente, sondern eine aktive Vite-Client- plus Express-Server-Architektur, datengetriebene MMO-Inhalte, Build- und Validierungswerkzeuge, Tests, Admin-Werkzeuge, Deploy-Skripte und Integrationspfade zu Firebase, Azure und AWS. Phase 2 dieser Dokumentation schärft deshalb nicht die Vision, sondern die praktische Arbeitsführung.

- Der aktive Laufzeitpfad ist laut Root-Skripten und Architekturhinweis: Vite-Client im Verzeichnis /client plus Express-/WS-Server im Verzeichnis /server.
- Das Repo ist bereits für einen demonstrierbaren Vertical Slice ausgerichtet: Content-JSON, Item-Registry, Quests, Inventory/Equipment, Tooltip- und Rendererpfade sind im Snapshot klar sichtbar.
- Das Projekt ist noch nicht öffentlich release-reif. Die größten Lücken liegen in Produktionsauth, finaler Datenbankverdrahtung, Asset-Bestückung, tiefer Balancierung, Tests über den Slice hinaus und operativer Release-Härtung.
- Für spätere Entwickler ist die wichtigste Regel: bestehende generische Runtime schützen, Content nur datengetrieben erweitern und jede Änderung gegen Validator, Build und Slice-Checks zurückspiegeln.

Empfohlene Leitlinie: Diesen Stand nicht als 'fertiges MMORPG', sondern als belastbaren Entwicklungsbaukasten mit schon erkennbarer Kernschleife behandeln. Genau darin liegt sein Wert.

2. Quellenbasis und Vorgehensweise

Bewertungsbasis waren die sichtbaren Dateien und Skripte des gelieferten ZIP-Snapshots. Daraus wurden Zählwerte, Laufzeitpfade, Build-Skripte, Game-Data-Dateien, Tests und hochsignifikante Modulfamilien extrahiert. Die Analyse priorisiert beobachtbare Implementierungen über Absichtsdokumente.

Drei Ebenen wurden getrennt beurteilt: erstens Konzept und Leitdokumente, zweitens tatsächlich vorhandene Runtime- und Content-Pfade, drittens Bereiche, die zwar als Modulbaum vorhanden sind, aber aus diesem Snapshot heraus noch nicht als shippable End-to-End-System belegt werden können.

Bereich	Dateien	Kurznotiz
Gesamtdateien	679	Snapshot Größe
Server	340	Runtime, Tests, Tools
Client	89	Renderer, UI, Networking
Dokumentation	84	Konzept- und Auditdokumente
Game-Data	56	Content, Manifest, Authoring

Integrationen	20	AWS/Firebase/Deploy
Admin/GM	10	Editor- und Operatortools

Die Größenordnung ist für ein 'nur-konzeptionelles' Repo zu hoch. Das Verhältnis von 340 Serverdateien, 89 Clientdateien, 84 Dokumentationsdateien und 56 Game-Data-Dateien spricht für ein Projekt, das bewusst als Baukasten aus Code, Daten und Handoff-Material aufgebaut wurde.

3. Aktive Runtime und Architekturwahrheit

Die wichtigste operative Wahrheit des Snapshots ist die Trennung zwischen aktiver Runtime und liegengebliebener Legacy-Struktur. Die Root-Skripte bauen Client und Server getrennt, der Startpfad führt in server/dist/index.js, und ARCHITECTURE_NOTE.md benennt die alte Next.js-Struktur ausdrücklich als inaktiv.

Pfad	Beobachtung	Bedeutung
package.json (Root)	build => client + server, start => server/dist/index.js	Root orchestriert, ist aber nicht selbst die aktive App
client/package.json	Vite-Client	Frontend-Layer für Renderer, UI, Networking
server/package.json	Validate + TypeScript Build	Backend mit Content-Gate im Build
ARCHITECTURE_NOTE.md	Next-Reste als Legacy markiert	Wichtig für neue Devs: nicht versehentlich falschen Startpfad pflegen

Kurz gesagt: Wer an diesem Repo arbeitet, darf Root nicht mit einer klassischen Next-App verwechseln. Für die aktuelle Laufzeit zählt Vite + Express + WebSocket. Die Legacy-Reste sind als technische Schulden zu behandeln, nicht als Führungsstruktur.

4. Code- und Datenanatomie des Snapshots

Die Server-Seite ist stark moduliert. Besonders dicht vertreten sind Welt-, Economy-, NPC-, Admin-, Persistence-, AI-, Dialogue- und Asset-Familien. Das bedeutet nicht automatisch Produktionsreife, wohl aber, dass die Architektur für künftige Tiefe vorbereitet wurde.

Server-Modulfamilie	Dateien	Einordnung
world	17	sichtbar im Snapshot
economy	13	sichtbar im Snapshot

npc	11	sichtbar im Snapshot
admin	8	sichtbar im Snapshot
persistence	7	sichtbar im Snapshot
ai	6	sichtbar im Snapshot
dialogue	6	sichtbar im Snapshot
structure	6	sichtbar im Snapshot
asset-brain	5	sichtbar im Snapshot
asset-registry	5	sichtbar im Snapshot
auth	5	sichtbar im Snapshot
events	5	sichtbar im Snapshot

Auf Client-Seite dominiert die UI-Schicht. Das ist plausibel, weil HUD, Panels, Tooltips, Editorhilfen und Statusflächen schnell viele einzelne Dateien erzeugen. Engine, Gameplay, State und Networking sind kleiner, aber vorhanden.

Client-Bereich	Dateien	Einordnung
ui	44	sichtbar im Snapshot
editor	8	sichtbar im Snapshot
state	8	sichtbar im Snapshot
gameplay	5	sichtbar im Snapshot
engine	4	sichtbar im Snapshot
assets	3	sichtbar im Snapshot
audio	2	sichtbar im Snapshot
networking	2	sichtbar im Snapshot
auth	1	sichtbar im Snapshot
localization	1	sichtbar im Snapshot

Die Game-Data-Schicht ist kein Beiwerk, sondern eine echte Arbeitsoberfläche. NPC-, Quest-, Dialog-, Item- und Spawn-Dateien liegen zentral und werden durch ein Manifest sowie eine Authoring-Guide-Datei flankiert. Daraus folgt: zukünftiger Content gehört primär in Daten, nicht in hart codierte Sonderfälle.

5. Implementierungsstand: Was wirkt tragfähig, was ist noch Scaffold?

Die folgende Matrix bewertet nicht die Vision, sondern den beobachtbaren Snapshot. 'Stark' bedeutet: aktive Pfade und konsistente Artefakte sind sichtbar. 'Brauchbar' bedeutet: der Bereich ist real, aber noch klar als Entwicklungsschicht zu lesen. 'Teilweise' oder 'Scaffold+' bedeutet: spürbar vorbereitet, aber nicht als releasefester Endzustand belegt.

Bereich	Status	Kurzbewertung
Build-/Startpfad	Stark	Root startet Vite-Client + Express-Server; Legacy-Next verbleibt als inaktiv.
WebSocket-/Tick-Kern	Stark	WorldTick, WS-Server und Client-Networking liegen als aktive Laufzeitpfade vor.
Quest/Dialog/Content	Stark	JSON-gesteuerte NPC-, Quest-, Dialog- und Spawn-Dateien plus Validator/Manifest.
Inventory/Equipment/Loot	Stark	Registry, Persistenz- und UI-Pfade sind im Snapshot vorhanden.
Persistenz	Brauchbar	Dateibasierte Slice-Persistenz sichtbar; Produktions-DB zusätzlich vorbereitet, aber nicht Endstand.
Renderer/UI	Brauchbar	Großer Renderer, HUD, Tooltip, Weltlabels und Inventar-UI vorhanden; weitere Politur offen.
Content Authoring	Stark	AUTHORING_GUIDE, validateContent, content-manifest, dedizierte Game-Data-Pfade.
CI/CD	Brauchbar	GitHub-Actions für Build/Test/Azure-Deploy vorhanden; Governance muss eng bleiben.
Produktionsauth	Teilweise	Firebase-/Server-Hooks vorhanden, aber kein belastbarer öffentlicher Release-Stand belegt.
DB/Infra	Teilweise	PG/Redis/Firebase/AWS-Adapter und Migrationen vorhanden, aber nicht als final verdrahtet belegt.
Welt-/Gesellschaftssimulation	Scaffold+	Viele Moduldateien vorhanden, aber Tiefe/Qualität pro Modul muss gezielt verifiziert werden.

Asset/GLB-Pipeline	Teilweise	Registry, Upload/Validation und Beispiel-GLBs vorhanden; echter Produktionscontent fehlt noch.

Wichtig ist die Differenz zwischen 'viele Dateien vorhanden' und 'live bis zum Ende bewiesen'. Für spätere Entwickler ist diese Ehrlichkeit Gold wert. Sie verhindert, dass Modulbäume mit echter Betriebsreife verwechselt werden.

6. Hochsignifikante bereits sichtbare Fähigkeiten

Ohne den Snapshot zu romantisieren, lassen sich mehrere echte Stärken nennen. Diese Abschnitte sind der Grund, warum sich ein Phase-2-Manuskript lohnt.

- Content-Validierung vor Server-Build: Der Server-Build ruft validateContent vor dem TypeScript-Compile auf. Fehlerhafte Daten sollen also nicht still in den Laufpfad rutschen.
- Automatisiertes Content-Manifest: game-data/content-manifest.json liegt bereits vor und zeigt, dass Content nicht nur lose herumliegt, sondern inventarisiert wird.
- Datengetriebene Kerninhalte: NPCs, Quests, Dialoge, Items und Spawns leben in JSON-Dateien. Das ist für No-Code- und Agentenarbeit extrem wertvoll.
- Reputation und Drop-Definitionen sind im Snapshot sichtbar. Das deutet auf bereits vertiefte Spielsysteme über einen reinen Hello-World-Slice hinaus.
- Renderer-/Interaction-Helfer: getClosestInteractable sowie Weltlabel-Funktionen und Renderer-Pfade zeigen, dass die Lesbarkeit der Welt bereits praktisch bearbeitet wurde.
- Tests und Testideen: 48 Server-Testdateien plus zusätzliche Markdown-Testskizzen zeigen ein Repo, das wenigstens versuchsweise verifiziert statt nur beschrieben wird.

Gleichzeitig gilt: Nicht jede im Dateibaum sichtbare Fähigkeit ist automatisch bis zur Demo- oder Release-Qualität gereift. Deshalb braucht jeder neue Entwickler einen harten Arbeitsmodus: erst lesen, dann validieren, dann klein ändern, dann erneut prüfen.

7. Onboarding-Leitfaden für spätere Entwickler

Ein guter Einstieg soll das Repo weder überfordern noch vereinfachen. Die folgende Reihenfolge hat sich aus dem Snapshot logisch ergeben.

- Schritt 1: Zuerst lesen – README_START_HERE.md, PROJECT_LOCK_RULES.md, final-lock/FINAL_TRUTH.md, docs/MASTER_DESIGN_BIBLE.md, docs/BUILD_SEQUENCE_MASTER.md, ARCHITECTURE_NOTE.md.
- Schritt 2: Danach aktive Runtime begreifen – Root-Skripte, client/package.json, server/package.json, server/src/index.ts, server/src/core/WorldTick.ts, client/src/engine/renderer.ts.
- Schritt 3: Content-Schicht lesen – game-data/AUTHORING_GUIDE.md, NPC-/Quest-/Dialogue-/Spawn-/Item-JSONs und das content-manifest.json.

- Schritt 4: Dann erst bauen und validieren – Content-Validator, Client-Build, Server-Build, danach Startpfad testen.
- Schritt 5: Änderungen immer nur klein und begrenzt vornehmen; neue Inhalte bevorzugt über Daten statt über Runtime-Sonderlogik hinzufügen.

Die wichtigste Schutzregel für neue Entwickler lautet: zuerst die Runtime-Wahrheit respektieren, dann Datenkonsistenz schützen, erst danach neue Features öffnen.

Aufgabe	Befehl	Zweck
Root bauen	npm run build	Baut Client und Server über Root-Skripte.
Server validieren	cd server && npm run validate	Prüft Content-Referenzen und generiert Manifest.
Server bauen	cd server && npm run build	Validierung + TypeScript-Build.
Client bauen	cd client && npm run build	Vite-Produktionsbuild.
Projekt starten	npm run start	Startet server/dist/index.js.
Root-Tests	npm run test	Vitest-Lauf auf Root-Ebene.

```
# typische erste Sitzung
cp .env.example .env
npm install
cd server && npm run validate
cd ../client && npm run build
cd .. && npm run build
npm run start
```

8. Release-Roadmap in fünf Wellen

Phase 2 soll nicht nur erklären, was da ist, sondern was als Nächstes vernünftig ist. Für dieses Repo ist Geschwindigkeit nur dann hilfreich, wenn sie in Gates gegliedert ist.

Welle	Fokus
1. Stabilisieren	Build, Validator, Smoke-Tests, Root-Startpfad, Vite/Express-Grenzen festziehen.
2. Vertical Slice absichern	Login/Name, Bewegung, Quest, Reward, Inventory, Equip, Loot, Save/Load reproduzierbar machen.
3. Content-Produktion skalieren	Neue NPCs/Quests/Dialoge nur noch über JSON und Validierung hinzufügen.

4. Produktionshärtung	Echte Accounts, DB-Verdrahtung, Abuse-Schutz, Performance, Monitoring, Recovery.
5. Release Candidate	Nur wenn Tests, Content-Audits, Deploy-Checklisten und operative Runbooks grün sind.

Der zentrale Denkfehler, den dieses Manuskript vermeiden soll: Ein funktionierender Slice ist nicht automatisch ein öffentlicher Release. Der Weg dazwischen ist Härtung, nicht Heldentum.

9. Interner Demo-Stand versus öffentliche Release-Reife

Aus diesem Snapshot lässt sich ein interner Demo-Stand plausibel begründen: Datengetriebener Content, Build- und Validierungsstruktur, sichtbare UI-/Renderer-Pfade, Inventory-/Item-Registry-Logik und mehrere Game-Data-Einträge deuten auf einen vorzeigbaren Kern. Für einen öffentlichen Release reichen diese Indizien jedoch nicht.

Kriterium	Interner Demo-Stand	Öffentlicher Release
Builds/Validator	sollten zwingend grün sein	müssen dauerhaft grün und reproduzierbar sein
Auth/Accounts	Test-/Entwicklerpfad tolerierbar	muss sicher, missbrauchsfest und recoverbar sein
Persistenz	Slice-Persistenz genügt	muss belastbar, migrationsfähig und mehrnutzerfest sein
Assets/Content	Placeholder akzeptabel	muss konsistent, komplett und rechtlich sauber sein
Ops/Monitoring	rudimentär möglich	Runbooks, Secrets, Backups, Alerts erforderlich

10. Hauptrisiken und Gegenmaßnahmen

Legacy-Struktur vs. aktiver Runtime-Pfad

Root enthält Next-Reste neben Vite/Express. Das ist tolerierbar, aber erklärungsbedürftig und aufräumungsreif.

Konzepttiefe > Verifizierte Runtime-Tiefe

Viele Module existieren als Dateibaum, aber nicht jedes System ist end-to-end als produktionsfest bewiesen.

Produktionsauth/Accountmodell

Für öffentliche Freigabe reicht Name-/Test-Login oder lose Firebase-Anbindung nicht aus.

Persistenz-Sprung

Dateibasierte Slice-Persistenz ist gut für Entwicklung, aber kein finales Mehrnutzerfundament.

Content-Wachstum

Ohne klare Autorenregeln kippt ein datengetriebenes MMO schnell in Referenz- und Balancing-Chaos.

Infra-/Kostenkontrolle

Azure/Firebase/AWS-Hooks sind vorhanden; ohne strenge Environment-Disziplin drohen teure Fehlkonfigurationen.

11. Arbeitsregeln für menschliche Devs und No-Code-/AI-Agenten

Dieses Repo schreit nicht nach Vollgas, sondern nach Disziplin. Gerade weil so viel vorbereitet ist, kann ein zu freier Agent mehr kaputtmachen als aufbauen.

- Jede Arbeitsrunde soll nur einen klar begrenzten Schritt bearbeiten.
- Vor Änderungen immer: Content-Validator, relevante Builds, vorhandene Tests und den aktuellen Slice-Stand kennen.
- Generische Runtime schützen. Neue Quests, NPCs und Dialoge nach Möglichkeit nur über Daten ergänzen.
- Unsafe AutoFixes nie direkt auf main. Harmloser Autofix nur auf Arbeitsbranch und danach Re-Check.
- Bei Inkonsistenz zwischen Item-Registry, Persistenz und Runtime immer zuerst Datenwahrheit klären, erst dann Politur.
- Legacy-Dateien nicht still entfernen, solange der aktive Runtime-Pfad nicht sauber dokumentiert und abgesichert ist.

Praktischer Merksatz: Dieses Projekt sollte wie ein Garten gepflegt werden – Unkraut raus, Struktur schützen, nichts mit der Kettensäge auf Verdacht umlegen.

12. Empfohlene nächste Arbeitsstränge

Auf Basis des Snapshots sind die nächsten vernünftigen Schritte nicht die größten, sondern die stabilsten.

Priorität	Empfehlung	Warum jetzt?
1	Validator-/Build-Gates streng halten	Sie schützen den datengetriebenen Baukasten vor schleichendem Verfall.

2	Vertical Slice reproduzierbar dokumentieren	Neue Entwickler brauchen einen festen Kern, nicht nur Versprechen.
3	Produktionsauth sauber planen	Das ist der große Release-Block, der nicht improvisiert werden darf.
4	DB/Infra real verdrahten	Dateibasierter Slice und vorbereitete DB-Module müssen sauber zusammenfinden.
5	Content-Autorenschicht weiter verstärken	Der größte Skalenvorteil dieses Repos liegt in data-driven Content.
6	Asset-/GLB-Qualität systematisch erhöhen	Der Schritt vom Scaffold zum spielbaren Eindruck hängt stark an Inhalten.

13. Handoff-Checkliste für spätere Entwickler

- Verstehe zuerst die aktive Laufzeitarchitektur, nicht die Legacy-Schale.
- Nutze den Validator als Pflichtstation, nicht als optionalen Luxus.
- Behandle game-data als erste Autorenschicht.
- Unterscheide strikt zwischen beobachtbar implementiert und nur vorbereitet.
- Arbeite branch-basiert; niemals 'mal eben' riskante Kernlogik auf main schieben.
- Dokumentiere jeden kleineren Fix so, dass der nächste Entwickler nicht archäologisch arbeiten muss.
- Wenn du unsicher bist, sei konservativ: Repo-Stabilität schlägt Aktionismus.

Dieses Manuskript soll späteren Entwicklerinnen, Entwicklern und Agenten nicht nur erklären, was das Projekt sein möchte, sondern vor allem, wie man daran arbeitet, ohne die bestehende Struktur zu zerschlagen.

Anhang A – Wichtige Snapshot-Marker

- GitHub-Workflows im Snapshot: main.yml, main_aremmoall.yml, main_wasd.yml
- Server-Tests: 40 TypeScript-Dateien plus 8 Markdown-Testskizzen
- Content-Manifest enthält aktuell 14 NPC-IDs, 12 Quest-IDs, 11 Dialogue-IDs und 26 Item-IDs.
- Game-Data sichtbar im Snapshot: 14 NPCs, 12 Quests, 11 Dialogeinträge, 26 Items.
- Besonders relevante Dateien für spätere Leads: ARCHITECTURE_NOTE.md, FINAL_CHECKLIST_AUDIT.md, FINAL_MASTER_DELIVERY_INDEX.md, docs/MASTER_DESIGN_BIBLE.md, docs/BUILD_SEQUENCE_MASTER.md, game-data/AUTHORING_GUIDE.md, server/src/tools/validateContent.ts.